

CNN 실습: Windows · VS Code · 5-fold 교차검증

실행 코드와 따라 하기 가이드 | R2 · 2026-09-29

R2 변경: 고양이·개 원본을 Oxford-IIIT Pet으로 교체하고, CIFAR-10만 총 10,000장으로 늘렸습니다. 설치 환경·모델·학습률·epoch·5-fold 절차는 R1과 같습니다. 정상인 기존 환경은 재설치하지 않습니다. 이번 제공 ZIP은 **CODE_ONLY(실제 이미지 미포함)**이며, 기존 MNIST 데이터는 재사용합니다. 교수자 데이터 준비 후 생성하는 WITH_DATA ZIP에는 실제 데이터가 들어갑니다. 제작 환경의 원본 서버 DNS 실패로 실서버 다운로드와 Windows GPU 실행은 미검증입니다.

이 가이드에는 CNN 기초 이론은 넣지 않았습니다. 설치, 데이터 준비, 네 가지 실습 실행, 결과 확인과 설정 변경만 다룹니다. 외부 CMD와 WSL2는 사용하지 않습니다. 명령이 필요한 설치 단계도 **VS Code 안의 Windows PowerShell**에서 진행합니다.

1. 이번 구성 한눈에 보기

실습	데이터	모델 또는 변경 조건	실행 파일
1-A: 기본 분류	MNIST	LeNet형 소형 CNN	01_mnist.py
1-B: 기본 분류	CIFAR-10	VGG형 소형 CNN	02_cifar10.py
1-C: 기본 분류	고양이 vs 개	MobileNet형 소형 CNN	03_catsdogs.py
2: 데이터 양 비교	MNIST	같은 모델, 학습 풀 200/400/800/1,600장	04_data_size.py
3: 모델 비교	CIFAR-10	VGG형 · ResNet18 · MobileNetV2 · DenseNet형	05_model_compare.py
4: 하이퍼파라미터 비교	MNIST	학습률·배치 크기·Dropout을 한 항목씩 변경	06_hyperparameters.py

모든 모델은 무작위 초기화에서 학습합니다. 사전학습 가중치를 자동으로 받지 않습니다. 마지막 요청의 네 실습만 남겼으며, 별도의 pretrained 모델 실습은 추가하지 않았습니다. VGG형 소형 모델은 VGG16 자체가 아니고, DenseNet형 소형 모델은 DenseNet121 자체가 아닙니다.

1.1 도메인별 장수와 교차검증

R1의 분할 비율과 5-fold 방식을 유지합니다. **MNIST·고양이/개는 총 2,000장, CIFAR-10만 총 10,000장**입니다. CIFAR-10의 10,000장은 학습 전용 장수가 아니라 학습과 시험을 합한 전체 장수입니다.

구분	MNIST	CIFAR-10	고양이 vs 개
준비할 실제 이미지	2,000	10,000	2,000
클래스 수	10	10	2
클래스별 이미지	200	1,000	1,000
고정 학습 풀	1,600	8,000	1,600
고정 시험	400	2,000	400
각 fold의 학습	1,280	6,400	1,280
각 fold의 검증	320	1,600	320

별도의 고정 train/val/test 이미지 폴더를 만들지 않습니다. 데이터셋 파일 하나와 분할 인덱스를 사용합니다. 다만 **교차검증을 하려면 각 fold에서 임시 검증 부분은 반드시 필요합니다.**

실행 흐름은 다음과 같습니다.

```

MNIST와 고양이/개:
2,000장 → 학습 풀 1,600장 + 고정 시험 400장

학습 풀 안에서 5-fold 반복
  fold 1: 1,280장 학습 / 320장 검증
  fold 2: 새 모델로 1,280장 학습 / 320장 검증
  ...
  fold 5: 새 모델로 1,280장 학습 / 320장 검증

각 fold의 최저 검증 loss epoch 기록
→ 다섯 epoch의 중앙값 선택
→ 새 모델을 학습 풀 1,600장 전체로 재학습
→ 이 단일 모델을 고정 시험 400장으로 평가

CIFAR-10도 같은 순서이며 장수만 다릅니다:
10,000장 → 학습 풀 8,000장 + 고정 시험 2,000장
각 fold: 6,400장 학습 / 1,600장 검증
최종 재학습: 8,000장 / 최종 시험: 동일 2,000장

```

5-fold는 하나의 모델을 이어서 다섯 번 학습하는 것이 아닙니다. fold마다 모델과 optimizer를 새로 만듭니다. 마지막에는 단일 최종 모델을 얻기 위해 한 번 더 재학습하므로, 조건 하나당 학습은 총 여섯 번입니다.

최종 시험(MNIST·고양이/개 400장, CIFAR-10 2,000장)은 새로 정의한 **교육용 내부 holdout**입니다. MNIST와 CIFAR-10의 공식 전체 test benchmark 점수가 아닙니다. 클래스 비율을 유지하는 StratifiedKFold를 사용합니다. 구현 근거는 문서 끝의 scikit-learn 공식 자료에 있습니다.

2. ZIP 압축 해제와 VS Code 준비

2.0 이미 R1을 설치한 교수자: 패치로 변경 파일만 적용

NGV_CNN_5Fold_PATCH_R1_to_R2.zip은 변경·추가 파일만 담았습니다. 먼저 실행 중인 Python 학습/다운로드를 종료합니다. 압축을 임시 폴더에 풀 뒤, **패치 안의 ngv_cnn_5fold 폴더 내용**을 기존 프로젝트의 같은 위치에 복사하고 동일 이름 파일만 덮어씁니다. 프로젝트 안에 ngv_cnn_5fold/ngv_cnn_5fold가 이중으로 생기면 안 됩니다.

패치에는 .venv, install.ps1, requirements.txt, .vscode 설정, 실제 NPZ·JSON, 모델·학습 엔진 파일이 없습니다. 기존 환경·데이터·학습 결과를 지우지 않습니다. 코드 내용을 직접 수정했던 경우에는 먼저 원본을 별도로 백업하고 CHANGES_R2.md와 R1_T0_R2.diff를 비교합니다.

정상인 기존 VS Code interpreter를 그대로 사용하고 **3장의 설치**는 건너뛰어 **4장의 00_build_data_bundle.py**부터 실행합니다. 빌더가 기존 데이터의 크기·출처를 검사합니다. MNIST는 유지하고, CIFAR-10 2,000장 또는 기존 고양이/개 데이터만 교체합니다. 새 데이터가 모두 준비되고 해당 쌍의 검사를 통과한 뒤, 교체 대상의 이전 NPZ·JSON을 teacher_sources/bundle_backups/ 아래에 보관합니다. 다운로드 실패 시 기존 쌍은 그대로 남습니다.

2.1 처음 시작할 때: 새 폴더에 압축을 풉니다

현재 파일 이름은 NGV_CNN_5Fold_CODE_ONLY_R2.zip입니다. Windows 탐색기에서 우클릭 → **모두 압축 풀기**를 선택합니다. 기존 실습 폴더에 덮어쓰지 않습니다.

예를 들어 다음 구조가 되도록 새 폴더를 사용합니다.

```
C:\Users\본인계정\AI_Class_5Fold\ngv_cnn_5fold\
install.ps1
00_check_environment.py
00_build_data_bundle.py
00_check_data.py
01_mnist.py
02_cifar10.py
03_catsdogs.py
04_data_size.py
05_model_compare.py
06_hyperparameters.py
07_collect_results.py
08_make_student_zip.py
09_run_checks.py
configs\
cnnlab\
data\
```

폴더의 실제 위치는 달라도 됩니다. 이후 실행 코드는 프로젝트 위치를 파일 자체에서 찾습니다. **ZIP을 탐색기에서 열어 보기만 한 상태로 코드를 실행하지 마세요.**

2.2 VS Code 설치와 확장 기능

Windows용 VS Code를 설치합니다. 공식 사이트는 문서 끝의 링크를 사용합니다. VS Code에서 Ctrl+Shift+X를 누르고 Microsoft의 **Python**과 **Python Debugger** 확장을 설치합니다.

VS Code의 **파일 → 폴더 열기**에서 install.ps1과 01_mnist.py가 바로 보이는 ngv_cnn_5fold 폴더를 엽니다. .vscode 폴더는 숨김처럼 보일 수 있지만 압축 파일에 들어 있습니다.

2.3 설치용 터미널 열기

터미널 → 새 터미널을 선택합니다. 이 프로젝트는 Windows PowerShell을 기본으로 설정했습니다. 다음처럼 PS가 보이면 맞습니다.

```
PS C:\Users\...\ngv_cnn_5fold>
```

다음을 실행합니다.

```
Get-Location
Get-ChildItem .\install.ps1
py -3.12 --version
```

현재 사용자 환경에서 이미 확인된 Python 3.12.10은 다시 설치할 필요가 없습니다. 새 PC에서 Python 3.12가 없을 때만 Windows용 Python 3.12 x64를 먼저 설치하고 VS Code를 다시 시작합니다.

이 가이드에는 `cd /d, %USERPROFILE%, set PYEXE` 같은 CMD 문법을 사용하지 않습니다. PowerShell에서 경로를 직접 이동해야 할 경우의 문법은 다음과 같습니다. 실제 압축 해제 위치와 맞을 때만 사용합니다.

```
Set-Location "$env:USERPROFILE\AI_Class_5Fold\ngv_cnn_5fold"
```

3. 패키지 설치와 Python 선택

3.1 NVIDIA GPU용 설치

VS Code 안의 PowerShell에서 먼저 실행합니다.

```
nvidia-smi
```

GPU와 드라이버가 확인되면 다음을 실행합니다.

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\install.ps1 -Compute cu128
```

이 명령은 프로젝트 내부 .venv를 만들고 다음 조합을 설치합니다.

```
Python 3.12
PyTorch 2.10.0
Torchvision 0.25.0
NumPy 2.3.5
scikit-learn 1.8.0
Matplotlib 3.10.8
Pillow 12.3.0
```

PyTorch와 torchvision의 조합 및 CUDA 12.8 wheel은 공식 이전 버전 설치 안내에 있는 조합입니다. 최신 버전을 뜻하는 것이 아니라 이번 코드에서 사용하는 고정 조합입니다. 나머지 버전은 코드 시험 환경에 맞췄습니다.

GPU용 PyTorch 패키지는 크기가 큼니다. 데이터 포함 ZIP을 만들더라도 Python 패키지 설치까지 오프라인이 되는 것은 아닙니다. WSL2의 TensorFlow 환경은 변경하지 않습니다. Windows 전역 Python에 직접 패키지를 설치하지도 않습니다.

3.2 GPU를 사용하지 않을 때

위 GPU 설치 명령 대신 다음을 실행합니다.

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\install.ps1 -Compute cpu
```

두 설치 명령을 연달아 실행하는 절차가 아닙니다. CPU도 동일한 실습을 지원하지만 모델 비교, 특히 ResNet18은 더 오래 걸릴 수 있습니다. 실습 시간을 특정 값으로 보장하지 않습니다.

3.3 이미 정상인 이전 Windows PyTorch 환경을 재사용할 때

설치를 반복하지 않고 이전 Windows .venv\Scripts\python.exe를 VS Code interpreter로 선택해도 됩니다. .venv 폴더를 복사하거나 이동하지 마세요. 이때는 위 설치를 건너뛰고 아래 환경 검사를 먼저 실행합니다. 누락 패키지나 버전 문제가 있으면 새 환경 설치 경로로 진행하는 것이 분명합니다.

3.4 VS Code interpreter 선택

Ctrl+Shift+P → **Python: Select Interpreter** → 이번 프로젝트의 아래 실행 파일을 선택합니다.

```
ngv_cnn_5fold\.venv\Scripts\python.exe
```

목록에 없으면 **Enter interpreter path**를 사용합니다. 기존 환경 재사용자는 그 환경의 원래 실행 파일을 선택합니다. WSL interpreter나 시스템 Python을 선택하지 않습니다.

3.5 실제 연산 검사

탐색기에서 00_check_environment.py를 엽니다. 우측 상단의 **Run Python File** 삼각형 버튼을 클릭합니다. Code Runner의 실행 명령이 아니라 Python 확장의 실행 명령을 사용합니다.

```
[PASS] Real convolution forward and backward succeeded.
```

이 메시지가 보여야 합니다. GPU 이름만 읽는 검사가 아니라 실제 convolution의 forward와 backward를 실행합니다. 출력된 python_executable이 의도한 환경인지도 확인합니다.

GPU 오류가 있으면 임의로 넘어가지 않습니다. 당장 CPU 경로를 확인하려면 파일 상단의 DEVICE = "auto"를 DEVICE = "cpu"로 바꾸어 검사합니다. 이후 실습 파일의 DEVICE도 CPU로 맞춰야 합니다. 이는 GPU 오류를 해결했다는 뜻이 아닙니다.

4. 실제 데이터 준비: Oxford-IIIT Pet + CIFAR-10 10,000장

현재 첨부 CODE_ONLY ZIP에는 실제 데이터가 없습니다. 처음 시작하면 이 절차가 필요합니다. 기존 정상 MNIST 쌍은 그대로 재사용합니다. 00_check_data.py는 파일이 없으면 분명하게 중단하며, 합성 데이터로 대신 진행하지 않습니다.

4.1 교수자 PC에서 원본을 한 번 준비하는 경로

00_build_data_bundle.py를 엽니다. 기본 설정은 다음과 같습니다.

```
ALLOW_DOWNLOAD = True
SOURCE_DIR = None
```

실행하면 실제 공개 원본에서 MNIST 2,000장, CIFAR-10 10,000장, Oxford-IIIT Pet 고양이/개 2,000장을 균형 있게 준비합니다. ALLOW_DOWNLOAD=True는 **지금 이 PC에서 다운로드를 허용한다는 명시적 선택**입니다. 이미 데이터가 든 ZIP이 아니라 는 점에 유의하세요.

MNIST의 다운로드·추출 방식은 R1 그대로입니다. 이미 data/mnist.npz와 data/mnist.json이 검사를 통과하면 다시 만들지 않습니다.

CIFAR-10: 공식 binary 압축파일을 스트리밍으로 읽고, 클래스별 고유 이미지 1,000장씩 확보하면 연결을 닫습니다. 원본 학습 배치 하나는 정확히 클래스 균형이 맞지 않을 수 있으므로 여러 배치에 걸쳐 필요한 후보를 읽습니다. 최종 NPZ는 정확히 **10,000장**입니다. 균형·중복 확인과 압축 해제에 선행 읽기 때문에 네트워크에서 읽은 이미지 후보 수가 10,000을 넘을 수 있습니다. **원본 전체를 받는 방식은 아니며 압축 데이터 읽기 상한은 64 MiB**입니다. 실패했다고 원본 전체 자동 다운로드로 전환하지 않습니다.

새 교수자 원본 캐시는 `teacher_sources/cifar_prefix_source_10000.npz`입니다. 기존 `cifar_prefix_source.npz`는 5,000개 레코드밖에 없어 10,000장 확보에 재사용하지 않지만 삭제하지도 않습니다. 완성된 원본 CIFAR binary/Python 압축파일을 가지고 있으면 그 파일에서 추출하고 다운로드하지 않습니다. 스트림 방식은 전체 공식 train 50,000장에서 균등 무작위로 추출한 것과는 다르며, 추출 방법을 JSON에 기록합니다.

고양이/개: 이제 Oxford-IIIT Pet의 공식 `images.tar.gz`와 `annotations.tar.gz`를 사용합니다. 이미지 출처만 바꾸며 기존 전처리(RGB, 중심부 종횡비 유지 맞춤, 96×96 , PIL bilinear)는 유지합니다. 공식 `annotations/trainval.txt`의 **species 열(세 번째 열)**을 읽어 $1 \rightarrow \text{cat}(0), 2 \rightarrow \text{dog}(1)$ 로 매핑합니다. 37개 품종 분류로 바뀌는 것이 아닙니다. 공식 `trainval`에서 고양이 1,000장·개 1,000장을 골라 기존 80:20 내부 분할을 적용합니다. 공식 Oxford test 이미지는 후보로 쓰지 않습니다.

중요: Oxford 전체 이미지·주석 원본 다운로드는 공식 안내 기준 약 **800 MB**입니다. 최종 2,000장만 저장하더라도 최초 원본 전송량까지 2,000장 크기가 되는 것은 아닙니다. 이 변경은 데이터 출처의 교체이며 다운로드 경량화가 아닙니다. 교수자가 한번 받고 NPZ·JSON만 학생에게 배포하세요. 진행량은 MiB로 출력되고, 서버가 Range 요청을 올바르게 지원하면 .part 다운로드를 이어받습니다. 공식 기본 호스트가 실패하면 같은 데이터의 다른 Oxford 공식 URL을 시도하며, 완성 원본은 공식 MD5로 검사합니다.

Oxford 출처·작성자·CC BY-SA 4.0 라이선스 링크·변경 내용(선별, 96×96 전처리, NPZ 재포장, 새 분할)은 `catsdogs.json`과 `data/README.txt`에 기록됩니다. 학생 배포 때 함께 유지합니다. 네트워크 다운로드는 제작 환경의 DNS 실패로 실서버 통합 검증을 완료하지 못했습니다.

4.2 원본 파일을 이미 갖고 있다면 다운로드하지 않습니다

원본을 다음 위치에 넣습니다.

```
teacher_sources\mnist.npz
teacher_sources\cifar-10-binary.tar.gz
teacher_sources\images.tar.gz
teacher_sources\annotations.tar.gz
```

CIFAR는 `cifar-10-python.tar.gz`도 지원합니다. MNIST는 torchvision 형식의 `MNIST\raw\train-images-idx3-ubyte` 및 `train-labels-idx1-ubyte` 파일도 지원합니다. 기존 작은 캐시를 복제하여 목표 장수를 채우지 않습니다. Oxford 원본을 이미 풀어 놓았다면 `teacher_sources/images/`와 `teacher_sources/annotations/trainval.txt`도 지원하며, torchvision의 `teacher_sources/oxford-iiit-pet/` 아래 동일 구조도 지원합니다. 이전 `cats_and_dogs_filtered.zip`을 이름만 바꾸어 넣으면 안 됩니다.

다른 원본 폴더를 지정할 수도 있습니다.

```
ALLOW_DOWNLOAD = False
SOURCE_DIR = r"C:\Users\본인계정\Downloads\dataset_sources"
```

`SOURCE_DIR`은 실제 폴더로 바꿉니다. 원본 파일이 부족하면 명확하게 중단합니다. 원본을 한 번 확보한 뒤에는 학생 PC마다 원본 다운로드를 반복할 필요가 없습니다.

4.3 준비 결과 검사

데이터 준비가 성공하면 아래 여섯 파일이 생깁니다.

```
data\mnist.npz
data\mnist.json
data\cifar10.npz
data\cifar10.json
data\catsdogs.npz
data\catsdogs.json
```

00_check_data.py를 열어 실행합니다. 도메인마다 다음 형식이 나와야 합니다.

```
[PASS] mnist: 2000 images | train pool=1600 | test=400
[PASS] cifar10: 10000 images | train pool=8000 | test=2000
[PASS] catsdogs: 2000 images | train pool=1600 | test=400
[READY] ...
```

파일 hash, 이미지 수, 클래스별 장수, 완전히 같은 픽셀의 중복, 분할 겹침을 검사합니다. 이 검사가 유사 이미지나 동일 동물의 다른 사진까지 완벽하게 탐지한다는 뜻은 아닙니다.

4.4 학생용 데이터 포함 ZIP 만들기

세 데이터 검사가 성공한 뒤 08_make_student_zip.py를 실행합니다. 생성 위치는 다음과 같습니다.

```
distribution\NGV_CNN_5Fold_WITH_DATA.zip
```

이 ZIP에는 코드와 실제 데이터 총 14,000장(MNIST 2,000 + CIFAR-10 10,000 + 고양이/개 2,000)이 들어갑니다. .venv, 원본 전체 압축파일, 학습 결과, 시험용 합성 fixture는 넣지 않습니다. 세 데이터셋 중 하나라도 없으면 이 ZIP을 만들지 못하도록 했습니다.

학생에게 WITH_DATA ZIP을 배포한 뒤에는 4.1~4.2의 다운로드를 생략하고 00_check_data.py만 실행하면 됩니다. 이 ZIP은 현재 응답에 첨부된 파일이 아니라, 데이터 확보에 성공한 교수자 PC에서 생성하는 파일입니다.

5. VS Code에서 실행하는 공통 방법

데이터 검사를 통과한 뒤 진행합니다. 앞으로는 긴 명령어를 입력하지 않습니다.

1. 실행할 .py 파일을 엽니다.
2. 우측 상단 **Run Python File**을 클릭합니다.
3. 터미널에서 Fold 1/5부터 진행되는지 확인합니다.
4. 완료 후 브라우저 보고서 또는 VS Code 이미지 미리보기를 봅니다.

.vscode/launch.json에도 같은 실행 구성을 넣었습니다. 실행 및 디버그 → 해당 파일 이름 선택 → Ctrl+F5로 실행해도 됩니다. 중단점을 사용하려면 F5입니다. 두 방식 중 하나만 사용합니다.

각 실행 파일 위쪽에는 다음 설정이 있습니다.

```
DEVICE = "auto"
OPEN_REPORT = True
CONFIG_FILE = "configs의 해당 JSON 파일 이름"
```

실제 CONFIG_FILE 값에는 configs/를 붙이지 않고 파일 이름만 들어갑니다. 기존 값을 그대로 두면 됩니다. auto는 사용 가능한 CUDA를 우선 선택합니다. GPU를 쓰지 않으려면 cpu로 바꿉니다.

보고서는 완료 시 브라우저에서 열립니다. VS Code 안에서 보려면 왼쪽 탐색기에서 결과 폴더의 PNG를 클릭하면 이미지 미리 보기가 열립니다. CSV도 VS Code에서 열 수 있고, Excel로 열어 수치를 비교할 수도 있습니다.

이 버전은 한 프로젝트에서 순차 실행합니다. 같은 GPU로 여러 실험을 동시에 돌리지 않습니다. 프로젝트 잠금이 동시 실행을 차단합니다. WSL2에서 다른 학습을 실행 중이라면 그것도 멈추어 자원 경쟁을 피하세요.

6. 실습 1: 세 도메인의 기본 분류

6.1 MNIST

01_mnist.py를 열어 실행합니다. 모델은 ReLU·BN을 사용하는 LeNet형 소형 CNN입니다. 원본 LeNet-5의 정확한 복제 모델은 아닙니다.

기본 설정은 configs/exp01_mnist.json입니다.

```
전체 이미지: 2,000
학습 풀: 1,600
5-fold마다 학습/검증: 1,280 / 320
고정 시험: 400
최대 CV epoch: 5
batch size: 32
learning rate: 0.001
```

터미널에는 fold별 epoch 결과가 나오고, 다섯 fold 뒤에 Final refit이 나옵니다. 완료되면 새 결과 폴더가 생성됩니다. 다음 이미지를 확인합니다.

```
train_loss.png
val_loss.png
train_accuracy.png
val_accuracy.png
confusion_test.png
confusion_test_normalized.png
confusion_oof.png
```

각 학습·검증 곡선에는 다섯 fold와 평균·표준편차가 들어갑니다. Confusion matrix의 행은 실제 정답, 열은 예측입니다. OOF 결과는 각 이미지가 자신의 검증 fold에 있을 때의 예측을 모은 진단 결과입니다.

6.2 CIFAR-10

02_cifar10.py를 열어 실행합니다. 설정은 configs/exp01_cifar10.json입니다. VGG형 소형 CNN, RGB 32×32, 최대 CV epoch 5는 그대로입니다. R2는 전체 10,000장 중 학습 풀 8,000장·고정 시험 2,000장을 사용합니다. 각 fold는 6,400장 학습·1,600장 검증입니다. 이 파일의 base.train_count만 1,600에서 8,000으로 바꾸었으며 다른 하이퍼파라미터는 동일합니다.

보고서에서 어떤 클래스 쌍이 혼동되는지 확인합니다. 비행기·자동차·새·고양이·사슴·개·개구리·말·배·트럭의 10개 클래스입니다. 데이터 장수만 늘렸으며 사전학습이나 epoch를 바꾸지는 않았으므로 높은 정확도를 보장하지 않습니다. 낮은 성능도 실제 결과 그대로 기록합니다.

6.3 고양이 vs 개

03_catsdogs.py를 열어 실행합니다. 데이터 출처는 Oxford-IIIT Pet이며, 설정 configs/exp01_catsdogs.json은 R1과 같습니다. depthwise separable convolution 기반의 작은 MobileNet형 CNN을 처음부터 학습합니다. 입력은 RGB 96×96, 최대 CV epoch는 8입니다.

MobileNetV3-Small의 사전학습 모델을 가져오는 실습은 아닙니다. 모형 이름은 코드에서 tiny_mobile입니다. 학습률·증강·학습 횟수와 데이터 수에 따라 결과가 달라집니다.

6.4 세 실습의 결과 폴더

예시는 다음과 같습니다. 날짜·시간과 끝 식별자는 실행마다 바뀝니다.

```
results\날짜시간_exp01_mnist_식별자\
  report.html
  comparison.csv
  status.json
  plan.json
  environment.json
  source_hashes.json
  mnist_lenet\
    report.html
    metrics.json
    config.json
    indices.json
    fold_metrics.csv
    history_cv.csv
    history_refit.csv
    per_class.csv
    test_predictions.csv
    final_model.pt
    fold_1_best.pt ... fold_5_best.pt
  examples\
    *.png
```

최상단 report.html은 실습 전체 요약입니다. 그 아래 조건별 report.html에 confusion matrix, 실제 이미지, 오분류와 곡선이 있습니다. 결과 폴더는 새로 만들어지므로 기존 학습 결과를 덮어쓰지 않습니다.

7. 실습 2: 데이터 양에 따른 성능 변화

04_data_size.py를 실행합니다. MNIST와 LeNet형 모델을 고정하고 학습 풀 크기만 바꿉니다.

조건	교차검증용 학습 풀	각 fold 실제 학습	각 fold 검증	최종 재학습	동일 시험
n0200	200	160	40	200	400
n0400	400	320	80	400	400
n0800	800	640	160	800	400
n1600	1,600	1,280	320	1,600	400

작은 집합이 큰 집합에 포함되는 **중첩 부분집합**을 사용합니다. 시험 400장은 매번 동일하고 test_hash도 같아야 합니다. 데이터 양에 따라 파일을 다시 다운로드하지 않습니다.

네 조건을 순차 실행한 뒤, 같은 보고서에서 다음을 비교합니다.

```
test accuracy vs 학습 풀 크기
test loss vs 학습 풀 크기
CV accuracy / CV loss vs 학습 풀 크기
최종 재학습 시간 vs 학습 풀 크기
5-fold 전체 처리 시간 vs 학습 풀 크기
동일 400장 시험 처리 시간 vs 학습 풀 크기
```

시험 이미지 수와 모델 구조가 같으므로 시험 시간은 크게 변하지 않을 수 있습니다. 작은 시간 차이는 측정 잡음·장치 상태의 영향일 수 있습니다. 데이터가 많을수록 정확도가 매번 단조롭게 증가한다고 가정하지 마세요.

학습량 목록은 configs/exp02_data_size.json의 variants에서 바꿉니다.

```
"variants": [
  {"name": "n0200", "train_count": 200},
  {"name": "n0400", "train_count": 400},
  {"name": "n0800", "train_count": 800},
  {"name": "n1600", "train_count": 1600}
]
```

MNIST에서 값은 10의 배수이고 50 이상, 1,600 이하여야 합니다. 400장 시험에서 이미지를 가져와 학습량을 늘리지 않습니다.

8. 실습 3: 동일 CIFAR-10 데이터로 네 모델 비교

05_model_compare.py를 실행합니다. 다음 네 조건을 순차 실행합니다.

코드 이름	모델	구분
tiny_vgg	VGG형 소형 CNN	교육용 자체 소형 모델, VGG16 아님
resnet18_cifar	ResNet18	32×32 입력에 맞게 첫 convolution과 pooling 수정
mobilenet_v2_half	MobileNetV2	width multiplier 0.5, 첫 convolution stride 수정
tiny_densenet	DenseNet형 소형 CNN	세 dense block, growth rate 12, DenseNet121 아님

R2의 모든 비교 조건에서 **학습 풀 8,000장·고정 시험 2,000장**을 사용합니다. `configs/exp03_models.json`의 `base.train_count`만 8,000으로 변경했습니다. 모든 조건에서 데이터셋 hash, 학습 풀, 시험 이미지, fold 분할, 입력 크기, optimizer 종류와 기본 학습률·최대 epoch를 유지합니다. 모두 `pretrained=False`입니다. 최종 재학습 epoch는 각 모델의 CV 결과로 정하기 때문에 달라질 수 있습니다.

확인할 지표는 정확도·loss뿐만 아니라 파라미터 수, 학습 시간, 시험 처리 시간입니다. 이 비교는 **동일한 기본 학습 설정 아래의 비교**이며, 모델마다 충분히 최적화한 최고 성능의 비교가 아닙니다.

설정은 `configs/exp03_models.json`에 있습니다. `variants`의 모델 이름은 위 네 코드 이름을 사용합니다. `width`는 자체 소형 모델의 설정이며 ResNet18의 전체 채널 수나 MobileNetV2의 `multiplier`를 일괄 변경하는 옵션은 아닙니다. MobileNetV2 `multiplier`는 `cnnlab/models.py`의 `width_mult=.5`에 있습니다.

9. 실습 4: 하이퍼파라미터 변화 비교

`06_hyperparameters.py`를 실행합니다. MNIST와 LeNet형 모델을 고정합니다. 기준 조건에서 한 항목씩 바꿉니다.

조건	학습률	배치 크기	Dropout	변경 내용
baseline	0.001	32	0.2	기준
lr_low	0.0003	32	0.2	학습률만 감소
lr_high	0.003	32	0.2	학습률만 증가
batch16	0.001	16	0.2	배치 크기만 감소
dropout50	0.001	32	0.5	Dropout만 증가

설정 위치는 `configs/exp04_hyperparameters.json`입니다. 예를 들어 다음 항목을 추가할 수 있습니다.

```
{"name": "weight_decay_high", "weight_decay": 0.001}
```

JSON에서는 마지막 항목 뒤에 불필요한 쉼표를 붙이지 않습니다. 조건 이름에는 영문·숫자·밑줄·하이픈을 사용하고, 중복된 이름을 만들지 않습니다.

기본값은 `base`, 조건별 변경값은 `variants`에 있습니다. `variants`에 지정된 키가 `base`보다 우선합니다. 학습률을 변경하려면 이번 코드에서는 **`learning_rate`**라는 키를 사용합니다. 이전 패키지의 `lr` 키와 섞지 않습니다.

바꾸고 싶은 값	키	수정 위치
학습률	<code>learning_rate</code>	해당 configs JSON
배치 크기	<code>batch_size</code>	해당 configs JSON
최대 CV epoch	<code>epochs</code>	해당 configs JSON
가중치 감쇠	<code>weight_decay</code>	해당 configs JSON
Dropout	<code>dropout</code>	해당 configs JSON
학습용 난수	<code>seed</code>	해당 configs JSON

바꾸고 싶은 값	키	수정 위치
증강 여부	augmentation	해당 configs JSON
학습 풀 수	train_count	해당 configs JSON
모델의 구조 자체	계층 코드	cnnlab/models.py

같은 epoch에서 배치 크기를 줄이면 optimizer 업데이트 횟수가 늘어납니다. 따라서 배치 크기 실험은 “동일한 업데이트 횟수” 비교가 아니라 “동일한 최대 epoch” 비교입니다.

여러 seed로 반복하려면 첫 결과가 끝난 뒤 `base.seed`를 42에서 43, 44로 바꾸어 다시 실행합니다. `cv_seed`, `subset_seed`와 데이터 JSON의 분할은 그대로 둡니다. 다섯 fold의 표준편차는 별도 seed 반복의 변동성이나 통계적 신뢰구간과 같지 않습니다.

10. 그래프와 표를 읽는 순서

먼저 `fold_metrics.csv`에서 다섯 fold가 모두 완료됐는지 확인합니다. 다음으로 `train_loss.png`와 `val_loss.png`를 함께 보고, `confusion_test.png`에서 오분류가 몰린 클래스를 확인합니다. `examples` 폴더에는 보고서에 사용한 실제 시험 이미지가 저장됩니다.

`metrics.json`과 `comparison.csv`의 핵심 항목은 다음과 같습니다.

항목	의미
<code>cv_accuracy_mean</code>	다섯 fold에서 선택한 checkpoint들의 검증 정확도 평균
<code>cv_accuracy_std</code>	그 다섯 정확도의 표본 표준편차
<code>cv_loss_mean</code>	선택된 checkpoint들의 검증 loss 평균
<code>selected_epochs</code>	각 fold 최저 검증 loss epoch의 중앙값
<code>test_accuracy</code>	전체 학습 풀로 재학습한 단일 모델의 고정 시험 정확도
<code>test_loss</code>	같은 모델의 고정 시험 cross-entropy loss
<code>test_macro_f1</code>	클래스별 F1을 같은 비중으로 평균한 값
<code>cv_total_seconds</code>	다섯 fold 학습·검증·중간 저장을 포함한 처리 시간
<code>refit_train_seconds</code>	최종 모델의 학습 epoch 처리 시간 합계
<code>total_fit_seconds</code>	CV부터 최종 모델 저장까지의 전체 처리 시간
<code>test_seconds</code>	고정 시험 처리 시간: CIFAR-10 2,000장, 나머지 400장
<code>test_ms_per_image</code>	시험 처리 시간을 실제 시험 이미지 수로 나눈 환산값

시간 측정은 GPU를 사용할 때 동기화를 포함합니다. 시험 측정 전에 입력만 사용하는 두 번의 warm-up을 수행합니다. 시험 시간에는 메모리에 이미 읽힌 이미지의 변환, 배치 전달, 모델 추론과 결과 수집이 포함됩니다. 원본 NPZ 읽기, 모델 초기 로딩, warm-up, 그래프 저장, 브라우저 출력은 제외합니다. **단일 이미지 실시간 latency나 카메라 FPS로 해석하지 않습니다.**

여러 조건에서 같은 시험 데이터를 반복 관찰하는 것은 요청한 교육용 비교입니다. 그 결과를 보고 계속 설정을 조정했다면 해당 시험은 더 이상 독립적인 최종 일반화 검증이 아닙니다. 코드에서 조건을 고르는 기준은 CV loss이며, test 정확도로 epoch를 고르지 않습니다.

11. 전체 결과 모으기

모든 실습이 끝난 뒤 07_collect_results.py를 실행합니다.

```
results\summary\all_results.csv
results\summary\report.html
```

CSV는 UTF-8 BOM 형식이므로 Excel로 열 수 있습니다. 필요하다면 Excel에서 **다른 이름으로 저장** → **.xlsx**로 저장합니다. 원래 실험 데이터는 CSV에 납니다. 이 패키지는 추가 Excel 라이브러리를 요구하지 않습니다.

CSV에는 완료된 모든 실행이 들어갑니다. 같은 설정을 여러 번 실행했으면 여러 행이 보이는 것이 정상입니다. 파일의 domain, dataset_sha256, test_hash, 장치와 seed를 함께 확인합니다. 서로 다른 도메인의 정확도를 한 줄로 줄 세워 모델 우열을 정하지 않습니다.

12. 코드 파일별 역할

파일	역할
00_build_data_bundle.py	교수자용 원본 준비 스위치
cnnlab/builder.py	원본 읽기·제한된 다운로드·균형 추출·배포 ZIP 생성
cnnlab/data.py	데이터 무결성·고정 80/20 분할·5-fold·중첩 부분집합
cnnlab/models.py	각 CNN의 실제 구조
cnnlab/engine.py	epoch 학습·평가·CV checkpoint 선택·최종 재학습
cnnlab/runner.py	실습 전체 실행 순서와 기록
cnnlab/reporting.py	그래프·confusion matrix·HTML·CSV
cnnlab/common.py	경로·seed·환경 검사·파일 저장·중복 실행 방지
01_...부터 06_...	VS Code에서 직접 실행할 실습 진입 파일
07_collect_results.py	완료 결과 통합
08_make_student_zip.py	데이터 검사 후 학생 배포용 ZIP 생성
09_run_checks.py	로컬 코드 기능 시험

cnnlab/engine.py의 logits = model(images), loss = loss_fn(logits, labels), loss.backward()에 중단점을 걸면 학습 입력과 출력을 볼 수 있습니다. F5로 실행하고 F10으로 진행합니다. 디버거로 중단하며 얻은 시간을 일반 실행 성능과 비교하지 마세요.

13. 오류가 발생했을 때

증상	조치
cd /d 또는 %USERPROFILE% 오류	CMD 문법을 섞은 경우입니다. 이 가이드의 PowerShell 문법만 사용합니다.
Python 모듈이 없다고 나옴	VS Code interpreter가 설치한 .venv인지 확인합니다.
Missing real dataset	현재 CODE_ONLY 배포에는 데이터가 없습니다. 4장의 원본 준비가 필요합니다.
원본 다운로드 DNS·timeout 오류	완성된 원본을 teacher_sources에 넣고 ALLOW_DOWNLOAD=False로 준비합니다.
원본 NPZ·ZIP 손상	불완전 원본 대신 정상 원본을 사용합니다. 학습 코드가 복제 데이터로 보충하지 않습니다.
CUDA 연산 실패	GPU 드라이버와 wheel 조합 확인 또는 CPU 경로로 명시적으로 전환합니다.
GPU 메모리 부족	다른 학습을 종료하고 해당 설정의 batch_size를 16으로 낮춥니다.
.running.lock 오류	다른 실습이 실행 중인지 확인합니다. 모든 관련 Python 프로세스가 종료된 것을 확인한 뒤에만 남은 잠금 파일을 지웁니다.
같은 설정을 바꿨는데 결과가 그대로임	JSON을 저장했는지, variants에 같은 키의 덮어쓰기 값이 있는지 확인합니다.
보고서가 자동으로 안 열림	해당 결과 폴더의 report.html 또는 PNG를 직접 엽니다.
실습이 오래 걸림	5-fold와 최종 재학습으로 조건당 여섯 번 학습합니다. 모델 비교를 순차 수행하고, 환경 확인용으로만 epochs를 1로 낮춥니다.

시험 중 중단하면 해당 실행은 interrupted_or_failed로 남습니다. 같은 파일을 다시 실행하면 처음부터 새 결과 폴더에서 시작합니다. optimizer 상태까지 복원하는 resume 기능은 넣지 않았습니다.

14. 실제 검증한 범위

제작 환경은 Linux CPU, PyTorch 2.10.0+cpu, torchvision 0.25.0+cpu입니다. **R1의 35개 기존 시험과 R2의 20개 추가 시험, 총 55개 단위·기능 시험을 통과했습니다.** 기존 시험 파일은 수정하지 않았습니다. 추가 시험은 CIFAR-10 10,000장·80:20·5-fold 수량, 여러 학습 배치에 걸친 제한 스트림, 캐시 재사용, Oxford species 파서·원본 MD5·로컬 폴더/압축파일 일치·백업을 검사합니다. 네트워크 동작은 실제 서버 대신 메모리 응답·실패 모의 입력을 사용했습니다.

R2 별도 합성 fixture로 기존 CIFAR-10 2,000장과 구형 고양이/개 쌍의 갱신, 이전 파일 백업, MNIST 파일 바이트 보존, 두 번째 실행의 무다운로드 재사용을 확인했습니다. CIFAR-10·고양이/개 두 실습의 5-fold → 최종 재학습 → 고정 시험 → 그림·CSV 저장을 작은 학습 폴과 1 epoch로 점검했습니다. CIFAR-10은 새 2,000장 holdout으로 검사했습니다. 학생 ZIP의 파일 구성과 백업·원본 제외도 확인했습니다. **실제 데이터셋의 정확도, 기본 전체 epoch 설정의 성능, Windows GPU 동작, 실서버 데이터 다운로드를 검증했다는 뜻은 아닙니다.** 합성 fixture와 그 결과 이미지는 사용자 배포 ZIP에 넣지 않았습니다.

현재 수정본은 CODE_ONLY임을 표지와 BUNDLE_STATUS.json에 명시했습니다. 실제 이미지나 시험용 합성 이미지는 제공 ZIP에 없습니다. CHANGES_R2.md, R1_TO_R2.diff, R2_CHANGE_AUDIT.json에 변경 내용을 기록했고, 환경 파일·모델 구조·학습 엔진과 R1/R2의 SHA256 동일 여부를 검사했습니다.

15. 실행 순서 체크

```

새 폴더에 ZIP 압축 해제
→ VS Code로 프로젝트 폴더 열기
→ Python 확장 설치와 Windows Python 3.12 확인
→ GPU용 또는 CPU용 패키지 설치 / 정상인 기존 환경 선택
→ 00_check_environment.py
→ [교수자, 현재 CODE_ONLY 배포] 00_build_data_bundle.py
→ 00_check_data.py
→ 01_mnist.py
→ 02_cifar10.py
→ 03_catsdogs.py
→ 04_data_size.py
→ 05_model_compare.py
→ 06_hyperparameters.py
→ 07_collect_results.py
→ [교수자, 데이터 준비 성공 시] 08_make_student_zip.py

```

공식 자료와 출처

본 문서의 실험 설계·파일명·설정은 이 패키지에서 새로 정한 것입니다. 다음은 사용 도구와 원본 데이터의 공식 출처입니다.

- PyTorch 이전 버전 설치 조합: <https://pytorch.org/get-started/previous-versions/>
- Python 가상환경: <https://docs.python.org/3.12/library/venv.html>
- VS Code Python 실행·디버깅: <https://code.visualstudio.com/docs/python/debugging>
- VS Code Windows 설치: <https://code.visualstudio.com/docs/setup/windows>
- Windows Python 3.12.10 설치 파일: <https://www.python.org/downloads/release/python-31210/>
- MNIST 공식 torchvision 문서: <https://docs.pytorch.org/vision/0.25/generated/torchvision.datasets.MNIST.html>
- MNIST 원본 NPZ: <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
- CIFAR-10 배포자 자료: <https://www.cs.toronto.edu/~kriz/cifar.html>
- Oxford-IIIT Pet 공식 데이터·다운로드·라이선스: <https://www.robots.ox.ac.uk/~vgg/data/pets/>
- Oxford-IIIT Pet torchvision 0.25의 species 매핑·공식 MD5: https://docs.pytorch.org/vision/0.25/_modules/torchvision/datasets/oxford_iiit_pet.html
- Oxford 이미지 직접 다운로드: <https://thor.robots.ox.ac.uk/datasets/pets/images.tar.gz>
- Oxford 주석 직접 다운로드: <https://thor.robots.ox.ac.uk/datasets/pets/annotations.tar.gz>
- MobileNetV2 공식 구현: https://docs.pytorch.org/vision/0.25/models/generated/torchvision.models.mobilenet_v2.html
- 교차검증·계층화 분할: https://scikit-learn.org/stable/modules/cross_validation.html

데이터의 권리와 이용 조건은 원배포자의 조건을 따릅니다. 공개 다운로드 URL이 무제한 상업 재배포 허가를 뜻하지는 않습니다. 본 패키지는 원 데이터에 대한 새로운 권리나 허가를 부여하지 않습니다.